

Frontend Interview Handbook — Part 3: Advanced React & Next.js

This part connects directly to what a search/AEO-focused company like ViewEngine will probe hardest.

SECTION A: RENDERING MODELS (CSR, SSR, SSG, ISR, RSC)

Q1: Explain CSR, SSR, SSG, and ISR — what problem does each solve?

Answer:

- **CSR (Client-Side Rendering):** Server sends a near-empty HTML shell; JS downloads, executes, and renders content in the browser. Fast to build, but slow initial content paint and poor for crawlers that don't fully execute JS or wait for it.
- **SSR (Server-Side Rendering):** Server renders full HTML on *every request*, sends complete content immediately. Good for personalized/frequently-changing data and SEO, but adds server load and response latency per request.
- **SSG (Static Site Generation):** HTML is rendered once at *build time* and served as static files from a CDN. Fastest possible delivery and best for crawlers, but content is only as fresh as the last build — not suited to per-user or rapidly changing data.
- **ISR (Incremental Static Regeneration):** Pages are statically generated but can be regenerated in the background after a defined interval or on-demand, without rebuilding the whole site. Combines SSG's speed with the ability to stay reasonably fresh — a strong fit for programmatic SEO pages built from a large, periodically-updated dataset.

Why this is the single most important question in this whole handbook for a ViewEngine interview: their business is literally "get your content found by crawlers and AI systems" — rendering strategy is the mechanism by which frontend architecture either helps or hurts that goal. Be fluent enough to explain this without notes.

Q2: Why does pure CSR hurt SEO and AI answer engine visibility specifically?

Answer: Traditional crawlers can execute JS but it costs more compute/time, so heavily JS-dependent pages get crawled less frequently or less completely — Google's own guidance has long recommended not relying solely on client-side rendering for critical content. AI answer engines (ChatGPT browsing, Perplexity, Google AI Overviews) generally favor fetching and parsing readily-available HTML content quickly; a blank shell that only fills in after JS execution is a worse candidate for being retrieved and cited accurately, especially under retrieval systems that may not run a full headless browser for every page they consider citing.

Follow-up: "So does that mean CSR is always wrong?" A: No — for content that doesn't need to be discovered/indexed (authenticated dashboards, internal tools, apps behind login), CSR is completely fine. The rule is: **anything meant to be found via search or cited by AI needs its core content in server-rendered or statically-generated HTML.**

Q3: What is hydration, and what problems can go wrong with it?

Answer: Hydration is the process where React "attaches" to server-rendered HTML in the browser — reusing the existing DOM nodes rather than re-rendering from scratch, and wiring up event listeners/interactivity. Common problems: **hydration mismatch** (server-rendered HTML doesn't match what the client would render — e.g., using `Date.now()` or `Math.random()` directly in render, or reading `window` during SSR) causes React to discard and re-render, hurting performance and sometimes causing visible flicker; and the delay between "content visible" (from SSR) and "content interactive" (after hydration completes) — a page can look ready but not respond to clicks yet, which is exactly what **INP** (a Core Web Vital) measures.

```
jsx
```

```
// classic hydration mismatch - server renders one thing, client renders another
function Timestamp() {
  return <span>{new Date().toLocaleTimeString()}</span>; // differs between server and client
}
```

Best practice: Guard non-deterministic or browser-only values with a `useEffect` that sets state after mount, so the server and initial client render match, and the "real" value fills in after hydration.

Q4: What are React Server Components (RSC), and how do they differ from traditional SSR?

Answer: RSC (used by Next.js App Router) let components render *entirely on the server*, never shipping their JS to the client at all — reducing bundle size for content that doesn't need interactivity. This is different from traditional SSR, which renders HTML on the server but *still* ships the full component JS to the client for hydration. With RSC, only components explicitly marked `"use client"` ship JS and hydrate; everything else stays server-only, meaning less JS downloaded, parsed, and executed — directly helping load performance and Core Web Vitals.

```
tsx
```

```
// Server Component (default in App Router) – no JS shipped to client
async function ProductList() {
  const products = await db.getProducts(); // can talk to DB/fetch directly
  return <ul>{products.map(p => <li key={p.id}>{p.name}</li>)}</ul>;
}

// Client Component – opts into shipping JS + hydration
"use client";
function LikeButton() {
  const [liked, setLiked] = useState(false);
  return <button onClick={() => setLiked(!liked)}>{liked ? '❤️' : '♡️'}</button>
}
```

Common mistake in interviews: Saying RSC and SSR are the same thing — they solve related but distinct problems (SSR = where HTML is generated; RSC = where component code lives and whether it ships to the client at all).

SECTION B: NEXT.JS — APP ROUTER & PAGES ROUTER

Q5: What's the difference between the Pages Router and App Router in Next.js?

Answer: Pages Router (`/pages` directory, the original Next.js model) uses file-based routing where each file is a page, with data fetching via `getServerSideProps`/`getStaticProps`/`getStaticPaths`, and all components hydrate on the client by default. App Router (`/app` directory, newer) is built around React Server Components by default, uses nested layouts, colocated loading/error states (`loading.tsx`, `error.tsx`), and fetches data directly inside async Server Components using `fetch` with built-in caching, rather than special exported functions.

	Pages Router	App Router
Directory	<code>/pages</code>	<code>/app</code>
Default rendering	Client-hydrated	Server Components by default
Data fetching	<code>getServerSideProps</code> / <code>getStaticProps</code>	<code>async</code> components + <code>fetch</code> caching
Layouts	Manual <code>_app.js</code> pattern	Native nested <code>layout.tsx</code>

Follow-up: "Which would you choose for a new project?" A: App Router — it's the current recommended direction and gives finer control over what ships to the client, which matters a lot for performance/SEO-focused work. Mention Pages Router familiarity is still valuable since a lot of production codebases haven't migrated yet.

Q6: In the App Router, how do you control caching/revalidation for a fetch, and how does this map to SSG/SSR/ISR concepts?

Answer:

```
tsx

// SSG-like: cached indefinitely at build/first request
fetch(url, { cache: 'force-cache' });

// SSR-like: always fetch fresh, no caching
fetch(url, { cache: 'no-store' });

// ISR-like: revalidate in the background every N seconds
fetch(url, { next: { revalidate: 60 } });
```

Why asked: Tests whether you understand that App Router unified the old three-function model (`getStaticProps`/`getServerSideProps`/ISR options) into fetch-level cache directives — a common point of confusion for people who only know the old model.

Q7: How does dynamic routing work in Next.js, and how would you use it for a programmatic SEO use case (thousands of generated pages)?

Answer: File-based dynamic segments — `[slug]/page.tsx` (App Router) or `[slug].js` (Pages Router) — map a URL parameter to a page template. For programmatic SEO at scale: use `generateStaticParams` (App Router) or `getStaticPaths` (Pages Router) to pre-render the most important/highest-traffic pages at build time, combine with ISR (`revalidate`) so the rest generate on-demand and get cached after first visit, rather than pre-building every single page upfront (which doesn't scale to tens of thousands of pages at build time).

```
tsx
```

```
// app/locations/[slug]/page.tsx
export async function generateStaticParams() {
  const topLocations = await getTopLocations(); // pre-build only the highest-v
  return topLocations.map(loc => ({ slug: loc.slug }));
}

export default async function LocationPage({ params }) {
  const data = await getLocationData(params.slug);
  return <LocationTemplate data={data} />;
}
```

This is very likely close to what ViewEngine's actual product infrastructure looks like — be ready to speak to it confidently.

Q8: How do you handle SEO metadata (title, description, Open Graph) per-page in Next.js App Router?

```
tsx

// static metadata
export const metadata = {
  title: 'Product Name | Brand',
  description: 'Concise, unique description under ~160 characters.',
  openGraph: { title: '...', images: ['/og-image.png'] },
};

// dynamic metadata (e.g., for programmatic pages)
export async function generateMetadata({ params }) {
  const item = await getItem(params.slug);
  return {
    title: `${item.name} | Brand`,
    description: item.summary,
  };
}
```

Why asked: Direct, practical test of SEO implementation skill, not just theory.

Q9: What's the difference between `getServerSideProps` and `getStaticProps` in the Pages Router (in case an older codebase uses it)?

Answer: `getStaticProps` runs at build time (or on-demand with ISR's `revalidate`), producing static HTML — fast, cacheable, best for content that doesn't need to be request-specific. `getServerSideProps` runs on *every request*, server-side, before the page renders —

necessary when content depends on request data (cookies, query params, user session) that can't be known at build time. Choosing `getServerSideProps` when `getStaticProps` + ISR would suffice is a common performance mistake, since it forces server rendering on every single request instead of serving cached static output.

Q10: How does Next.js Image (`next/image`) help with Core Web Vitals, and what does it actually do under the hood?

Answer: `next/image` automatically: serves responsively-sized images based on viewport (via `srcset`), lazily loads offscreen images by default, converts to modern formats (WebP/AVIF) when supported, and requires explicit `width`/`height` (or `fill` with a sized parent), which prevents CLS since the browser reserves space before the image loads.

tsx

```
import Image from 'next/image';  
<Image src="/hero.jpg" alt="Hero banner" width={1200} height={600} priority />
```

`priority` disables lazy-loading for above-the-fold images critical to LCP — a good detail to mention since it shows practical, not just textbook, knowledge.

Rapid-Fire Round

- **Q: What's the difference between `next/link` and a regular `<a>` tag?** A: `next/link` enables client-side navigation (no full page reload) and prefetches the linked page's code in the background for instant navigation, while still rendering a real `<a>` tag under the hood (so crawlers can follow it normally).
 - **Q: What is a layout in App Router, and why does it matter for performance?** A: A `layout.tsx` wraps child pages and persists across navigations within it — it doesn't re-render or re-fetch on every route change, avoiding redundant work for shared UI (nav, sidebar).
 - **Q: What's streaming SSR / `Suspense` for?** A: Lets the server send HTML in chunks as it becomes ready rather than waiting for the entire page to finish rendering — the user sees fast-loading parts immediately while slower data-dependent parts stream in, improving perceived performance.
 - **Q: What's the difference between `loading.tsx` and a manual loading spinner?** A: `loading.tsx` is a Next.js App Router convention that automatically wraps a route segment in `Suspense`, showing that fallback while the segment's data loads — no manual state management needed.
 - **Q: Can Server Components use `useState`/`useEffect`?** A: No — those require client-side interactivity, so any component using hooks must be marked `"use client"`.
-

End of Part 3. Say "next part" for Part 4: SEO, Technical SEO, AEO & GEO.